Matura — Praktische Arbeit

Dauer: 5 Stunden (4h Praxis + 1h Theorie) **Hilfsmittel:** Visual Studio offline, NuGet-Pakete (offline-Cache), eigene Notizen / Code-Sammlung **Nicht erlaubt:** Internet, KI-Tools

Aufgabe	Punkte
1. AnimSkript	/ 35
2. RoutenPlaner	/ 35
3. Theorie	/ 30
Gesamt	/ 100

Punkte	Note
0 – 50	5
51 – 63	4
64 – 75	3
76 – 88	2
89 – 100	1

Die Aufgaben können in beliebiger Reihenfolge bearbeitet werden. Schreib deinen Namen auf jedes Blatt und in jede Solution (`README.txt`).

Aufgabe 1 — AnimSkript (35 P)

Ziel

Du sollst einen kleinen Interpreter für die Sprache **AnimSkript** programmieren. AnimSkript zeichnet einen farbigen Linienzug auf einem WPF-Canvas. Der Quelltext wird in einer TextBox eingegeben, mit einem Klick auf "Run" tokenisiert, geparst und ausgeführt.

Beispielprogramm

```
POSITION 100 100
FARBE Blau
DICKE 2

WIEDERHOLE 4 MAL {
    BEWEGE 60 0
    BEWEGE 0 60
    BEWEGE -60 0
    BEWEGE 0 -60
}

WENN POSX > 200 DANN {
    FARBE Rot
    BEWEGE 0 100
}
```

Erwartetes Ergebnis: ein blauer Quadratstapel ausgehend von (100, 100), gefolgt von einer roten Senkrechten, falls die aktuelle X-Position > 200 ist.

Sprachelemente

Element	Beschreibung
`POSITION x y`	Setzt die aktuelle Cursor-Position. **Kein Strich** wird gezeichnet.
`BEWEGE dx dy`	Bewegt den Cursor relativ um `(dx, dy)` und zeichnet dabei einen **Strich** in aktueller Farbe und Dicke.
`FARBE name`	Setzt die aktuelle Stiftfarbe. Erlaubt: `Schwarz`, `Rot`, `Gruen`, `Blau`, `Gelb`.
`DICKE n`	Setzt die aktuelle Stiftdicke (1 ≤ n ≤ 10).
`WIEDERHOLE n MAL { ... }`	Wiederholt den Block n-mal.
`WENN <vergleich> DANN { ... }`	Bedingte Ausführung. Optional mit `SONST { ... }`.
`<vergleich>`	`POSX`, `POSY` oder `SCHRITT` `<` / `>` / `==` zahl

`SCHRITT` ist die Anzahl der bisher ausgeführten `BEWEGE`-Anweisungen.

Schlüsselwörter sind GROSS geschrieben. Zahlen sind ganze Zahlen (positiv oder mit Vorzeichen `-`). Beliebig viele Leerzeilen / Spaces / Tabs sind erlaubt.

Im Angabeprojekt vorhanden

In `Angabeprojekt_AnimSkript/` findest du:

- Funktionsfähige WPF-Oberfläche (Code-TextBox, Run-Button, Canvas, Fehler-ListBox)
- Klasse `Token` mit `TokenType`-Enum (alle Typen vorgegeben)
- Abstrakte Klasse `Expression` mit `Parse(...)` und `Run(...)`-Signatur
- Klasse `Programm`-Stub
- Eine `Stiftzustand`-Klasse (Position / Farbe / Dicke / Schrittzähler) — fertig
- Eine `Zeichner`-Klasse, die ein `Stiftzustand`-Objekt + Canvas zum Zeichnen einer Linie verwendet — fertig
- Stubs werfen `NotImplementedException`

Teilaufgaben

1. Tokenizer (6 P)

Im `MainWindow.xaml.cs` findest du `private void btnRun_Click(...)`. Befülle dort die `tokens`-Liste, indem du den Quelltext analysierst und für jedes erkannte Token einen `Token` mit korrektem `TokenType` und `Value` erzeugst.

Erwartetes Verhalten:

- Alle Schlüsselwörter werden als entsprechende `TokenType` erkannt
- Ganzzahlen (auch mit führendem `-`) werden als `Number` erkannt
- Geschweifte Klammern `{` und `}` werden als `LKL` / `RKL` erkannt
- Unbekannte Zeichenketten erzeugen einen `Error`-Token, der **nicht** zum Absturz führt, sondern in der Fehler-ListBox angezeigt wird

2. Programm- und Statement-Parser (12 P)

Die Klasse `Programm` parst den vollständigen Quelltext, indem sie wiederholt eine konkrete Statement-Klasse für jeden Anweisungstyp aus dem Token-Strom liest. Implementiere folgende Statement-Klassen (Vorschlag, eigene Namen erlaubt):

- `PositionStmt`, `BewegeStmt`, `FarbeStmt`, `DickeStmt`
- `WiederholeStmt`, `WennStmt`

Erwartetes Verhalten:

- Jede Statement-Klasse erbt von `Expression`
- `Parse(...)` zieht die zugehörigen Tokens vom Anfang der Liste
- `Run(...)` führt die Aktion am übergebenen `Stiftzustand` / `Zeichner` aus
- Verschachtelte Blöcke (`WIEDERHOLE`, `WENN`, `SONST`) werden korrekt rekursiv geparst

3. Vergleichsausdruck und WENN/SONST (5 P)

Implementiere einen einfachen Vergleichsausdruck (`POSX` / `POSY` / `SCHRITT` Operator Zahl), der zur Laufzeit den aktuellen Wert aus dem `Stiftzustand` liest und einen `bool` zurückgibt. Die `WennStmt` führt je nach Ergebnis den DANN- oder SONST-Block aus.

4. Auswertung mit Canvas-Ausgabe (8 P)

Wenn alle Statements geparst sind, ruft `btnRun_Click` `programm.Run(...)` auf. Erwartet:

- Das Canvas wird vor jedem Run **geleert**
- Jede `BEWEGE`-Anweisung zeichnet eine `Line` mit aktueller Farbe + Dicke
- `POSITION` ändert nur die Cursor-Position, ohne Strich
- `WIEDERHOLE` führt seinen Block n-mal aus
- `WENN` evaluiert den Vergleich und führt den passenden Block aus

5. Fehlerbehandlung (4 P)

Mind. drei verschiedene Fehlertypen werden in der Fehler-ListBox angezeigt, ohne dass das Programm abstürzt:

- Tokenisierungsfehler (unbekanntes Zeichen)
- Parser-Fehler (z. B. Zahl erwartet, fehlende `{`)
- Laufzeitfehler (z. B. unbekannte Farbe)

Aufgabe 2 — RoutenPlaner (35 P)

Ziel

Du erweiterst eine vorgegebene Client-Server-Anwendung. Der **Server** kennt ein Stationsnetz (Stationen + Verbindungen) und beantwortet zwei algorithmische Anfragen:

- **Kürzester Pfad** zwischen 2 Stationen (Dijkstra)
- **Beste Tour** über bis zu 6 ausgewählte Stationen (Permutationsgenerator + Pfadlängen)

Der **Client** zeigt das Netz auf einem Canvas und erlaubt es, Pfade/Touren anzufragen sowie das Ergebnis lokal in einer SQLite-Datenbank zu speichern und wieder zu laden.

Im Angabeprojekt vorhanden

In `Angabeprojekt_RoutenPlaner/` findest du eine fertige Solution mit drei Projekten:

- **`NetworkLib/`** — Klassen `Transfer`, `MSG` (Enum `MsgType`, alle Felder), `Receiver`-Interface. Komplett fertig.
- **`Server/`** — Konsolen-App mit fertigem `TcpListener`, fertiger Stations-/Verbindungsdatensammlung (im Code) und einem `ServerReceiver`, der eingehende `MSG`s entgegennimmt. Methoden `BerechneKuerzestenPfad(...)` und `BerechneBesteTour(...)` werfen `NotImplementedException`.
- **`Client/`** — WPF-App mit fertigem Canvas-Rendering der Stationen + Verbindungen, fertigen ComboBoxen, ListBox, Buttons (Click-Handler werfen `NotImplementedException`). linq2db-Models `RouteEintrag` und `RoutenDb` sind vorhanden, ebenso die SQLite-DB `routen.db`.

NuGet-Pakete `linq2db` (6.2.1) und `Microsoft.Data.Sqlite` (10.0.7) sind im Client referenziert.

Teilaufgaben

1. Verbindung zum Server (4 P)

Im Client gibt es einen Button **"Verbinden"** mit Click-Handler `btnVerbinden_Click`. Befülle ihn:

- TCP-Verbindung zum Server (Default `127.0.0.1:5050`) öffnen
- `Transfer` instanziieren, `Start()` aufrufen
- Eine `MSG` mit `MsgType.InitRequest` senden
- Statustext aktualisieren

2. Stationen befüllen (4 P)

Wenn der Server eine `MSG` mit `MsgType.InitAnswer` schickt, übernimm die Stationen + Verbindungen in den Client und ruf die fertige Methode `Redraw()` auf. Sobald die Listen befüllt sind:

- ComboBoxen `cbVon` und `cbNach` mit allen Stationen befüllen
- ListBox `lbTour` mit allen Stationen befüllen (für Mehrfach-Auswahl)

3. Kürzester Pfad — Dijkstra (9 P)

Im **Server** implementiere die Methode `DijkstraSolver.Solve(int vonId, int nachId)`. Erwartet wird:

- Korrekte Implementierung mit Prioritätswarteschlange (`PriorityQueue<,>`) oder gleichwertiger Struktur
- Rückgabe: `(List<int> path, double total)` mit allen IDs in Reihenfolge und der Gesamtdistanz
- Falls kein Pfad existiert: Rückgabe einer leeren Liste, Distanz `double.PositiveInfinity`

Im **Client** befülle den Click-Handler `btnPfad_Click`:

- Sendet `MSG` mit `MsgType.PathRequest`, `FromId` / `ToId` aus den ComboBoxen
- Beim Empfang von `MsgType.PathAnswer`: Pfad **rot** auf dem Canvas hervorheben (Linien zwischen den IDs), Distanz im Statustext

4. Beste Tour — Permutationen (8 P)

Im **Server** implementiere `PermutationenSolver.Solve(List<int> ausgewaehlteIds)`. Erwartet:

- **Eigener** Permutationsgenerator (rekursiv oder iterativ), **nicht** über LINQ-Bibliothek
- Erste übergebene ID ist der **fixe Startpunkt** der Tour
- Pro Permutation wird die Summe der Pfadlängen (Dijkstra zwischen aufeinanderfolgenden Stationen) gebildet
- Rückgabe: `(List<int> ordnung, double total)` mit der besten gefundenen Reihenfolge

Im **Client** befülle `btnTour_Click`:

- Liest die ausgewählten IDs aus `lbTour` (max. 6, sonst Fehlertext)
- Sendet `MSG` mit `MsgType.TourRequest`
- Beim Empfang von `MsgType.TourAnswer`: Tour **blau** zeichnen, Stationen **fortlaufend nummerieren** (1, 2, 3, ...), Gesamtdistanz im Statustext

5. Speichern in SQLite per linq2db (5 P)

Über das Menü **Route** → **Speichern**: Dialogfenster, das einen Namen abfragt, dann den aktuell angezeigten Pfad oder Tour mit `linq2db` in die Tabelle `RouteEintraege` speichert (`Id`, `Name`, `Typ` (Pfad / Tour), `StationsIds` (CSV-String), `Distanz`).

Erwartet:

- Name darf nicht leer sein und nicht doppelt vorkommen → Fehlermeldung
- Nach erfolgreichem Speichern: Statustext-Update + ListBox `lbGespeichert` aktualisieren

6. Laden aus SQLite (3 P)

Über das Menü **Route** → **Laden**: Dialogfenster mit allen gespeicherten Routennamen aus der DB. Bei Auswahl wird die gespeicherte Route am Canvas wieder angezeigt (rot für Pfad, blau für Tour, Nummerierung), Distanz im Statustext.

7. GUI-Vorlage (2 P)

Diese Punkte gibt es für die korrekte Verwendung der vorgegebenen GUI-Elemente (Canvas, ListBox, ComboBoxen, Statustext, Menü). Wenn der Schüler eine völlig andere GUI baut, gibt es 0 Punkte für diesen Punkt — die vorgegebenen Steuerelemente sollen weiterverwendet werden.

Pflichtprüfung (auch unabhängig von der Punkteverteilung)

- [ ] Server startet, gibt Stationsliste auf der Konsole aus
- [ ] Client verbindet sich, zeigt Stationen + Verbindungen
- [ ] Pfad zwischen 2 Stationen wird korrekt berechnet und visualisiert
- [ ] Tour über 4 Stationen wird korrekt berechnet, beste Reihenfolge nummeriert eingezeichnet
- [ ] Speichern + Laden funktioniert mit SQLite-DB

Aufgabe 3 — Theorie (30 P, ca. 60 min)

> *Bezüge auf deine eigenen Implementierungen aus Aufgabe 1 und Aufgabe 2 sind ausdrücklich erwünscht.*

Block A — Programmiersprachen & Parser (8 P)

A1. ABNF-Grammatik (3 P)

Schreibe in **ABNF**-Notation die Grammatik der `WIEDERHOLE`-Anweisung deines AnimSkripts (inklusive der inneren Statements als Verweis auf `<Anweisung>`). Verwende `::=`, `|`, `{ }`, `[ ]`, `( )` korrekt.

A2. Chomsky-Hierarchie (2 P)

In welche **Chomsky-Klasse** (Type 0/1/2/3) ordnest du:

- die Tokenisierung deines AnimSkripts ein? Begründe.
- die Statement-Grammatik deines AnimSkripts ein? Begründe.

A3. Tokenisierung (1 P)

Welche Tokens entstehen aus folgender Zeile? Liste pro Token Typ und Wert auf:

```
WIEDERHOLE 4 MAL { BEWEGE -10 20 }
```

A4. AST-Baum zu einem Beispielausdruck (2 P)

Zeichne den AST (Syntaxbaum) für folgendes AnimSkript-Fragment:

```
WIEDERHOLE 3 MAL {
    BEWEGE 10 0
    WENN POSX > 50 DANN {
        FARBE Rot
    }
}
```

Block B — Graphentheorie & Algorithmen (10 P)

B1. Begriffe am Stationsnetz (3 P)

Gegeben ist folgendes 6-Knoten-Stationsnetz (Skizze auf dem Angabezettel — Knoten A, B, C, D, E, F mit gewichteten Kanten):

- **(½ P)** Wie hoch ist der **Grad** des Knotens C?
- **(½ P)** Ist der Graph **bipartit**? Begründe.
- **(½ P)** Welcher Knoten ist der **zentrale Knoten** des Graphen? (Definition: kleinster maximaler Abstand zu allen anderen Knoten)
- **(½ P)** Welcher Knoten würde bei seinem Ausfall den Graphen in zwei Teilgraphen zerlegen? Begriff?
- **(½ P)** Wie viele Kanten enthält der **aufspannende Baum** des Graphen mindestens?
- **(½ P)** Erkläre den Unterschied zwischen einem **Weg**, einem **Kreis** und einem **Zyklus**.

B2. Dijkstra-Trockenlauf (4 P)

Führe Dijkstra vom Knoten **A** aus dem Graphen aus B1 per Hand aus. Trage in eine Tabelle pro Iteration ein: welcher Knoten wird finalisiert, aktuelle Distanzen, Vorgänger.

B3. Permutationen — Komplexität (1 P)

- Wie viele Permutationen entstehen für **N = 6**?
- Wie viele für **N = 8**?
- Begründe in einem Satz, warum die Aufgabe `N ≤ 6` auf 6 begrenzt ist.

B4. Pseudocode rekursiver Permutationsgenerator (2 P)

Schreibe Pseudocode für einen **rekursiven** Permutationsgenerator, der eine Liste übergeben bekommt und alle Anordnungen mit fixiertem ersten Element erzeugt.

Block C — UML & Software-Pattern (8 P)

C1. Klassendiagramm zu Aufgabe 1 (4 P)

Zeichne ein UML-Klassendiagramm der Parser-Hierarchie deines AnimSkripts. Mindestens enthalten:

- Abstrakte Basisklasse `Expression`
- Mind. 5 konkrete Statement-Klassen
- Beziehung zu `WiederholeStmt` und `WennStmt` mit ihren Kindern (Composite-Pattern)
- Sichtbarkeiten (`+` , `-` , `#`)

C2. Software-Pattern erkennen (3 P)

Welche **Software-Pattern** finden sich in deinen Aufgaben? Nenne genau drei und ordne sie konkret zu:

- Pattern 1 → wo in deinem Code? Was macht es?
- Pattern 2 → wo? Was macht es?
- Pattern 3 → wo? Was macht es?

(Erlaubt: Composite, Interpreter, Observer, Singleton, Iterator, Factory.)

C3. UML-Beziehungstypen (1 P)

Nenne und unterscheide kurz die folgenden UML-Beziehungstypen:

- Aggregation
- Komposition
- Generalisation
- Abhängigkeit (Dependency)

Block D — Netzwerk, ORM, Threads, XML (4 P)

D1. XML-Serialisierung — Regeln (2 P)

Nenne **drei Regeln**, die eine Klasse erfüllen muss, damit sie mit `XmlSerializer` serialisiert werden kann. Was ist das **Polymorphie-Problem** bei XML-Serialisierung und wie löst man es typischerweise?

D2. Threads (1 P)

Wie viele Threads laufen typischerweise:

- in deinem Client (bei einer Verbindung)?
- in deinem Server (bei zwei verbundenen Clients)?

Begründe kurz, **warum** für jede TCP-Verbindung ein eigener Thread sinnvoll ist.

D3. ORM und Datenbinding (1 P)

- Was bedeutet **ORM** und wozu wird es in Aufgabe 2 verwendet?
- Wann brauche ich in WPF **Datenbinding**? Welche Schnittstelle muss eine Klasse implementieren, damit sich die UI bei Änderungen aktualisiert?

Allgemeine Hinweise

- Beide Praxisaufgaben **müssen kompilieren**, ansonsten gibt es 0 Punkte für die jeweilige Aufgabe.
- Wenn du eine Teilaufgabe nicht schaffst, **schreib in die** `**README.txt**` der Solution, was funktioniert und was nicht — das gibt Teilpunkte.
- Lieber eine kleine, robuste Lösung als eine ambitionierte, die nicht läuft.
- Schreib deinen Namen oben auf jedes Theorie-Blatt.

Viel Erfolg!